SyncTalk: Real-Time Messaging & WebRTC Audio/Video Calling

Master Technical Handbook: Beginner to Extreme Senior Architecture

Project: SyncTalk Enterprise Collaboration
Engineer: Akshay Chandran P
Tech Stack: Flutter, Node.js/TypeScript, Socket.IO, WebRTC, Redis Pub/Sub, Hive Encrypted Outbox, FCM Data Wakeup

---

Section 1: End-to-End Architecture & Working Explanation

1.1 The Split-Brain Architecture

SyncTalk separates communication channels based on latency and payload requirements: 1. **REST API:** Handles authentication, user profile management, contact discovery, and presigned media uploads. 2. **Socket.IO (WebSocket):** Handles real-time text messaging, typing indicators, presence, and WebRTC signaling. 3. **WebRTC (Peer-to-Peer):** Carries direct encrypted voice and video media streams using SRTP. 4. **Redis Pub/Sub:** Bridges Socket.IO clusters horizontally across multiple backend nodes.

1.2 WebRTC Peer Connection Handshake (STUN & TURN)

1. **Signaling:** Client A sends an **SDP Offer** (Session Description Protocol) to Client B over Socket.IO.
2. **STUN (Session Traversal Utilities for NAT):** Contacts public Google STUN servers to discover its public IP and port (NAT Hole Punching).
3. **ICE Candidates:** Interactive Connectivity Establishment candidates are gathered and exchanged via signaling.
4. **TURN (Traversal Using Relays around NAT):** If both users are behind symmetric corporate firewalls that block P2P connections, media traffic falls back to a TURN relay server.

1.3 Offline Message Delivery & Hive Outbox Queue

When a user sends a message in airplane mode: 1. The message is assigned a UUID, timestamp, and status `SENDING` (single clock icon). 2. Saved immediately to an encrypted **Hive** local box. 3. Added to a FIFO memory queue. 4. When Socket.IO reconnects, the queue flushes messages sequentially. The server responds with `ack(messageId, serverTimestamp)` transitioning the status to `SENT` (single check), then `DELIVERED` (double checks), and finally `READ` (blue checks).

(Handbook continues with WebRTC audio ducking, FCM background wakeup, and end-to-end encryption mechanics)